全流程图解 · PIPELINE GUIDE

悬停旋翼设计管线

从 4 个输入数字到一整套可制造图纸：翼型族 → 2D 升阻表 → 3D 自由尾迹 → 配平 → CMA-ES 寻优的全链路流程图与模块说明书。每个模块的逻辑与算法用人话讲清，所有行话第一次出现都当场解释。

2026-10-25 · 配套《验证报告（人话版）》

目录

一、这个项目到底是干嘛的（30 秒版本） 2

二、全流程总图 2

三、3D 求解器的内循环 4

四、名词小词典（浓缩版） 4

五、模块说明书 5

airfoil.py — 翼型几何工具箱（304 行） 5

famgen.py — 翼型一家人生成器（65 行） 6

panel2d.py — 2D 无粘面板法（398 行，mini-XFOIL 的心脏） 6

bl2d.py — 边界层积分法（192 行） 6

foil2d.py — 2D 总装车间（136 行） 6

kernels3d.py — 涡的数学发动机（76 行） 6

rotor3d.py — 3D 自由尾迹求解器（439 行，重头戏） 6

design_opt.py / design_opt2.py — "基因"翻译官（194/261 行） . . . 6

opt_cmaes.py — CMA-ES 优化器（194 行） 7

finalize.py — 交付车间（134 行） 7

bench2.py / run_bench.py — 考试系统 7

verify_light / heavy / figs.py — 审计团队 7

六、一次评估的完整旅程（时间去哪了） 7

七、三档尾迹：快慢之间的取舍 7

八、红线自证：为什么敢说"没用 BEMT" 7

九、文件地图与快速上手 8

一、这个项目到底是干嘛的（30 秒版本）

想象有人给你提了一个要求："帮我设计一台直径 2.08 米的'电风扇'，要能往上吹出 200 公斤的力，只用 2 片叶子，叶子后缘厚度必须是 2 毫米，而且悬停时要尽量省电。"

这个程序接单后全自动做完五件事：①自己画出叶片每个截面的翼型形状（机翼剖面）；②自己决定叶片的平面形状（哪里宽哪里窄）和扭转角（叶根到叶尖各拧多少度）；③用自研气动求解器算出"这么设计要耗多少电"；④反复微调形状，一代一代试，直到电耗降不下去；⑤输出一整套图纸和性能报告。

> 输入是 4 个数字，输出是一套可制造的图纸。全程不需要任何商业软件（无 CAE/CFD 许可证），只依赖 numpy / scipy / numba / matplotlib 和一台普通电脑，完整寻优一晚上跑完。

二、全流程总图

从 4 个输入数字到交付图纸的完整链路。核心循环是：优化器出一组"基因"（设计向量）→ 翻译成桨叶形状 → 算出耗电 → 优化器总结经验出下一组。防假货机制（每 8 代高保真抽查）在右路把关。



图 1：端到端主流程（含优化循环与防假货抽查）

三、3D 求解器的内循环

3D 求解器是一个"鸡生蛋"问题：环量决定尾迹，尾迹诱导风速，风速决定迎角，迎角又决定环量。解法是不动点迭代 + 松弛：先猜一份环量，循环"算诱导风→查表得升阻→反推新环量"，直到前后两次环量几乎不再变化。

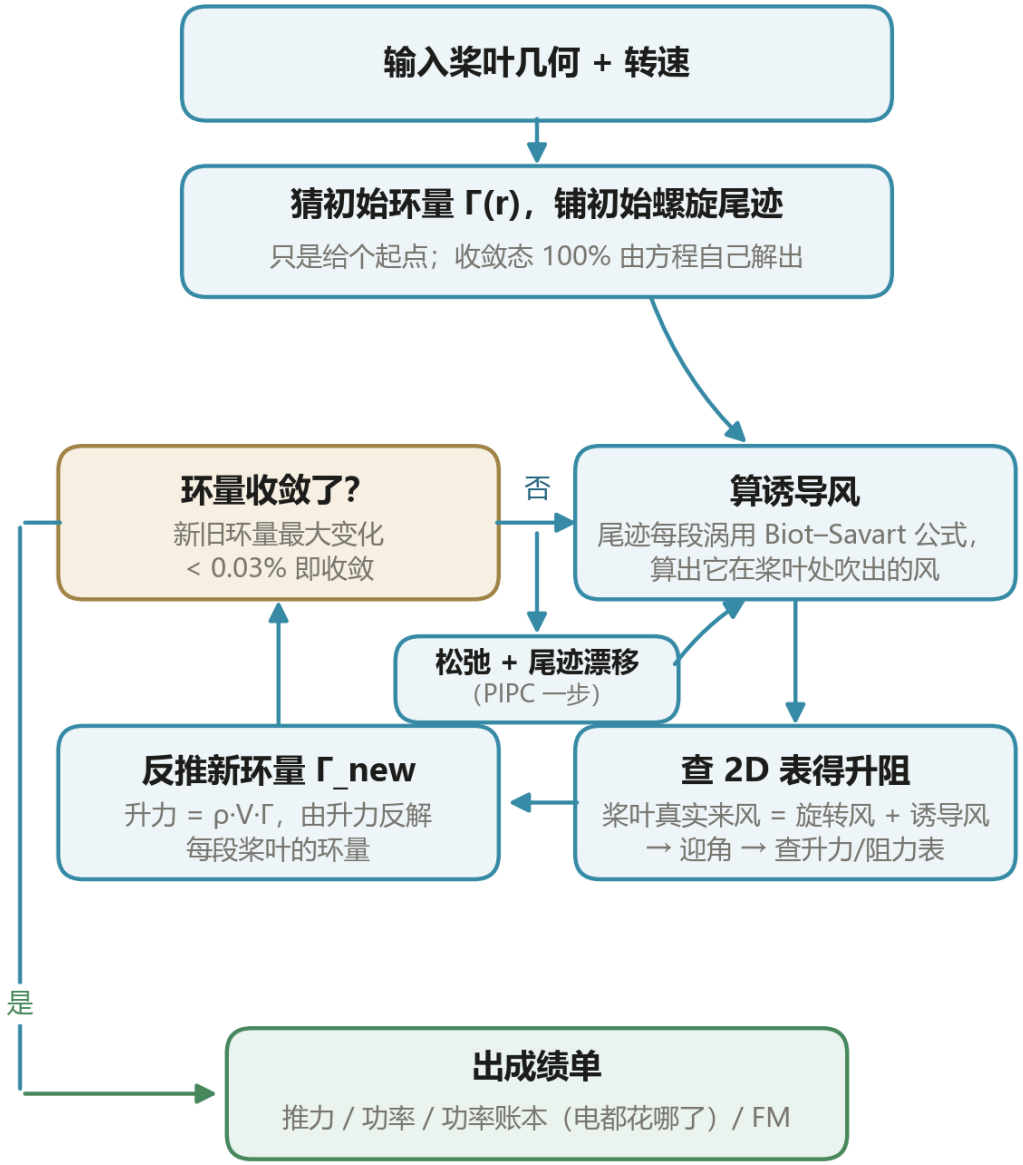


图 2：自由尾迹内循环（尾迹形状不预设，自由漂移收敛）

四、名词小词典（浓缩版）

完整版见 docs/PIPELINE_GUIDE.md 第 2 节。

4.1 旋翼零件与性能指标

名词	人话解释
弦长 c / t/c / 弯度	翼型截面的宽度 / 最厚处除以弦长 / 中弧线弓起的高度（弓得越狠升力越大阻力也大）
扭转 / 总距	叶片从根到尖逐渐"拧"的角度（叶尖转得快需要迎角小）/ 整片桨一起拧的基准角（直升机的油门）
实度 σ	所有叶片面积 ÷ 桨盘面积，"叶子盖住圆面的比例"
CT / CQ / FM	推力、扭矩的无量纲系数（让大小不同的桨可比）/ 悬停品质因数 = 理想最小功率 ÷ 实际功率，即"离物理满分有多近"，本项目唯一目标
配平	拧总距直到拉力正好 = 200 kgf

4.2 项目自造的词

名词	人话解释
三档尾迹	尾迹追多细追多远的三个固定档位：fast≈13s/次（优化内环）、hi-fi≈25s/次（抽查）、ultra≈57s/次（终验）——像看烟圈：帧率越高跟得越久越真实，也越费时间
φ=1.2（入流修正）	尾迹切成格子算会导致桨叶处下洗风系统性低估，对实验扫描后乘 1.2 修正。管推力读数
κ_cal=1.2（功率定标）	所有算例扭矩被系统性低估 15~25%，诱导功率整体乘 1.2 让读数贴近实验。管功率读数
κ_B（已删）	κ_cal 的前任"毯子因子"，无条件 ×1.2，实为重复惩罚，已删除
Cd×1.15 硬修正	自研 2D 阻力比实验低 12~18%，表格里直接乘 1.15 防优化器追假低阻
瓦特账本	功率拆成 P_axial（压空气下行的功）+ P_swirl（尾流打转动能）+ P_prof（表面摩擦），每瓦有去向、无隐藏系数
幻影最优	粗档里找到的"超级好设计"，细档复核就露馅（通常把失速藏在粗模型看不见的地方）
Gate A/B/C	自定的三道验收关卡（模型准/够快/达标），每道预先写好止损条款

4.3 求解器黑话

名词	人话解释
BEMT（被禁）	教科书标配的"叶素-动量理论"，快但算不准桨尖涡与尾迹收缩。本项目红线：禁止当求解器
自由尾迹	不预设尾迹形状，让每段涡按当地风速自由漂移，自己收敛成真实的收缩螺旋
Biot－Savart / 涡核	"一根通了风的导线在旁边吹多大风"的物理定律 / 给涡一个软芯半径，避免中心速度爆炸
桨尖涡 / 环量 Γ	叶尖甩出的最强漩涡（悬停直升机下洗的烟圈）/ 每段桨叶"背着"的升力强度，升力 = ρ·V·Γ
PIPC	尾迹节点漂移的数值格式（预测-校正+超松弛），悬停自由尾迹经典算法（Bagai－Leishman 1995）
面板法 / 边界层	把翼型表面切成小段贴涡解方程（升力）/ 贴壁薄空气层的摩擦与分离（阻力与失速）
CST 参数化	用一组可调系数描述任意翼型形状的数学语言，"翼型的基因"
CMA-ES / Nelder－Mead	一群候选每代撒出去试、表现好的方向拉大撒网椭圆的进化优化器 / "一个盲人拿三角形下山"的简单寻优法（7 维时代老将）

五、模块说明书

airfoil.py — 翼型几何工具箱（304 行）

定义"翼型是什么"（一圈封闭坐标点，钝尾缘=首尾两点留 2mm 开口），提供 NACA 教科书生成器、CST 参数化生成器、余弦重采样（前尾缘自动加密）、旋转平移、读写 .dat、算最大厚度/弯度/前缘半径。上层模块只需要"一圈坐标点"这一种表达。

famgen.py — 翼型一家人生成器（65 行）

沿展向长出一族互相衔接的翼型：站位 r → 厚度按 tc(r)=tc_root+(tc_tip−tc_root)·r^1.2 插值；当地弦长 c（米）→ 尾缘相对开口 = 0.002÷c 注入 CST 尾缘项。所以无论弦长多少，尾缘开口永远是 2 毫米——硬约束逐站位精确执行（12 站位实测全过，见验证报告）。已知瑕疵：弯度标定偏 27%，待重标定。

panel2d.py — 2D 无粘面板法（398 行，mini-XFOIL 的心脏）

把翼型表面切成 ~160 个小段（面板），每段贴一条强度线性变化的旋涡；"空气不能穿进表面"给出方程组，尾缘补 Kutta 条件（上下表面到尾缘速度相等）定出唯一解。三个诀窍：矩阵只和几何有关，分解一次后每个迎角 O(N) 秒解；预解 0° 和 90° 两个"单位来流"，任意迎角线性组合；钝尾缘补源+涡组合面板消角点奇性。可压缩性用 Kármán－Tsien 修正。

bl2d.py — 边界层积分法（192 行）

算摩擦阻力和分离（失速）。三段接力：Thwaites 层流段从驻点一步步积分动量厚度 → Michel 判据（何时层流撑不住变湍流）→ Head 湍流段（"卷吸方程"，边界层像滚雪球把外流卷进来）推进到尾缘。阻力 = Squire－Young 在尾缘收割动量损失 + Hoerner 钝尾缘底阻。历史坑（已解决）：湍流方程刚转捩时数值爆炸→改守恒形式；前缘分离泡误判失速→泡后重置形状因子。

foil2d.py — 2D 总装车间（136 行）

输入一个翼型，输出整条极线（一串迎角的 Cl/Cd/Cm）。流水线：面板法解无粘 → 沿表面从驻点"行军"算边界层 → 边界层的排挤效应换算成小"吹气"速度重解一次面板法（粘性-无粘耦合）→ 升力=环量×ρV，阻力=摩擦+底阻 → 分离则升力打折并标记失速。约 3 ms/迎角。

kernels3d.py — 涡的数学发动机（76 行）

一段直线涡在任意点激起的风速：精确有限段 Biot－Savart 公式加涡核正则化（涡核→0 时严格退化回教科书解；核有限时中心速度有界）。Numba JIT 多核并行，每秒几十亿"段×点"对——这是"自由尾迹能塞进优化环"的物理前提。

rotor3d.py — 3D 自由尾迹求解器（439 行，重头戏）

桨叶用环量分布 Γ(r) 描述；环量被叶尖叶根"甩"出去，沿尾缘拖出螺旋状涡面，叶尖那圈最强=桨尖涡。尾迹不预设形状：每个涡节点按当地合速度自由漂移（PIPC 预测-校正+松弛 1.3+步长限幅），近处 3.5~9 圈自由演化、更远处冻结、最外圈解析螺线延长。主循环"载荷↔环量"不动点迭代 22~34 次，松弛 0.35 防振荡，环量变化 <3e-4 收敛。三个工程稳定器（防数值毛刺非物理假设）：诱导速度限幅、展向三点平滑、弦向 3 条采样线平均。输出推力/功率/瓦特账本/无量纲系数。

design_opt.py / design_opt2.py — "基因"翻译官（194/261 行）

17 维设计向量 = 桨尖速度 + 扭角 6 控制点单调样条 + 弦长 5 控制 PCHIP + 弯度 3 点 + 根/尖厚度。配平：总距 8° →18° 粗扫后割线细调到拉力误差 <2~3%（每个候选要完整重跑 2~4 次 3D 求解，耗时大头）。约束（违反罚 1e6）：桨尖马赫 ≤0.52、根部厚度 ≥12%、升力 ≤0.85×Cl_max。极线缓存按（截面,Re,M）为键，命中率 86%——同族翼型反复出现时 2D 计算几乎

免费。

opt_cmaes.py — CMA-ES 优化器（194 行）

每代撒 14 个"探险者"试不同基因组合，表现好的方向把下一代的"撒网椭圆"（均值+协方差）挪过去、拉过来，自动放大缩小探索范围。多保真调度：内环全用 fast 档（~30-45 秒/候选含配平），每 8 代把当前最优拉到 hi-fi 档重考——fast 档成绩作数与否以 hi-fi 为准。每个候选完整记录到 JSONL 可续跑；独立监工每 30 分钟巡检、崩溃自动重启、连崩 3 次熔断。

finalize.py — 交付车间（134 行）

最终高保真复核 + 输出图纸：41 站位几何 CSV、12 个站位翼型 .dat（真实弦长、文件名含 2mm 标识）、三维叶面点云（可进 CAD）、performance.json 全性能明细。

bench2.py / run_bench.py — 考试系统

硬编码 Caradonna-Tung（悬停基准之王，与设计同规格）、Martin-Tung、Harrington、DJI 9450 的几何与实验数据，自动跑分输出误差矩阵。数据源 CSV 与配套基准仓库逐位核对一致。

verify_light / heavy / figs.py — 审计团队

第三方审计自动化后盾：light（秒级）涡核公式 vs 解析解、FM 恒等式、数据源逐位核对、κ 扫描；heavy（~4 分钟）复跑基准核心工况与交付叶片对答案；figs 重画验证报告全部 11 张图。

六、一次评估的完整旅程（时间去哪了）

步骤	干什么	耗时
1	边界检查：设计向量出界直接判 100 万分出局	~0
2	翻译：17 个数 → 扭角/弦长/弯度曲线 → 18 个站位翼型	毫秒级
3	每站位 2D 极线（52 迎角）：缓存命中则 0，未命中 ~0.2 秒/站位	0~2 秒
4	配平第 1 次求解：3D 自由尾迹 fast 档（22 轮外迭代）	~13 秒
5	配平第 2~4 次求解（割线逼近 200 kgf）	~13-40 秒
6	约束检查（失速/马赫/厚度）	毫秒级
7	记录到 JSONL，新最优则更新 best 文件	毫秒级

表 5：单候选评估流水线（fast 档实测 ~30-45 秒；一次完整寻优 = 40 代 × 14 候选 ≈ 一夜）

七、三档尾迹：快慢之间的取舍

档位	每步转角 dψ	追几圈	单次求解	用在哪
fast（快档）	10°	4.5 圈	~13 秒	优化内环，跑几千次的那种
hi-fi（高保真档）	7.5°	6 圈	~25 秒	每 8 代抽查优化结果
ultra（超深档）	5°	10 圈	~57 秒	最终交付前算总账

表 6：尾迹分辨率三档

八、红线自证：为什么敢说"没用 BEMT"

红线原文：禁止用 BEMT（叶素-动量理论）及其一维动量变体作为求解器。本项目的实现：①每一个点的诱导风都来自"尾迹上所有涡段 + 所有桨叶附着涡"的 Biot–Savart 直接求和——没有任何一步用动量公式求入流；②尾迹收缩、下沉、桨-涡干扰、叶尖损失全部是自由尾迹解出来的结果，且与实验对账通过（桨尖涡转 180° 后半径 0.897R vs 实验 0.905R，差 0.9%）；③动量理论唯一出场的地方是 FM 的分母（理想功率下限）——那是拿教科书公式当"满分线"来打分，不参与任何流场求解。

九、文件地图与快速上手

src/rotorgen/：airfoil.py famgen.py（翼型几何+族） · panel2d.py bl2d.py foil2d.py（2D 链） · kernels3d.py rotor3d.py（3D 链）
scripts/：design_opt.py design_opt2.py（参数化+配平） · opt_cmaes.py（寻优） · finalize.py（交付） · bench2.py run_bench.py（基准） · verify_light.py verify_heavy.py verify_figs.py（审计）
results/：基准成绩、优化轨迹、交付物 docs/：本文档、验证报告、FM075 计划

```
pip install -r requirements.txt # Python 3.12+; 仅 numpy/scipy/numba/matplotlib/cma
python scripts/run_bench.py # 跑基准考试（对答案）
python scripts/design_opt2.py # 冒烟：评估种子设计一次
python scripts/opt_cmaes.py # 端到端寻优（约一夜）
python scripts/verify_light.py # 秒级自检
```

想深入某个专题：所有失败与修复的完整记录 → RESEARCH_AND_BENCHMARK_LOG.md；每个数字准不准、哪些坑还没填 → VERIFICATION_REPORT.pdf（人话版）；FM 0.75 目标的路线图 → docs/FM075_PLAN.md。